EX.NO: 1	LINUX KERNAL COMPILATION
DATE:	

Aim:

To upgrade the kernel file of the linux operating system using kernel compilation process.

Apparatus Required:

1. Linux OS
2. Virtual Machine

Description:

Linux kernel compilation provides the user to unlock the features which are not available for the standard users. Linux compilation process allows the users to modify their kernel depending on their hardware and software application environment. Following points shows the steps involved in compiling a kernel.

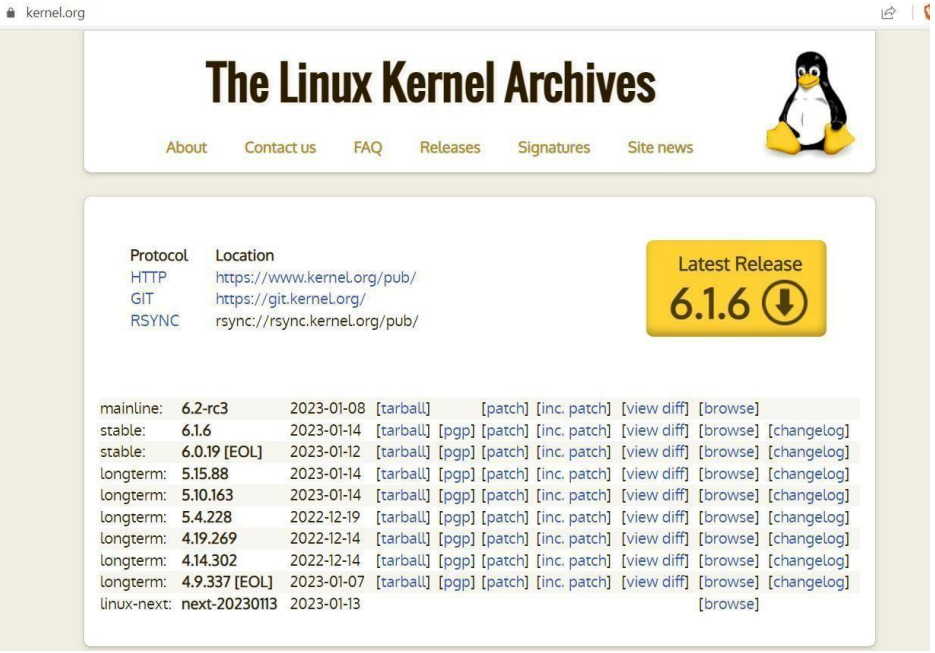
Building Linux Kernel:

The process of building a Linux kernel can be performed in seven easy steps. However, the procedure may require a significant amount of time to complete, depending on the system speed.

Follow the steps below to build the latest Linux kernel.

Step 1: Download the Source Code

1. Visit the official kernel website and download the latest kernel version. The downloaded file contains a compressed source code.



2. Open the terminal and use the wget command to download the Linux kernel source code:

wget https://cdn.kernel.org/pub/linux/kernel/v6.x/linux-6.0.7.tar.xz

The output shows the “saved” message when the download completes.

```
ubuntu@ubuntu: ~
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu@ubuntu:~$ wget https://cdn.kernel.org/pub/linux/kernel/v6.x/linux-6.0.7.tar.xz
--2023-01-09 07:13:58--  https://cdn.kernel.org/pub/linux/kernel/v6.x/linux-6.0.7.tar.xz
Resolving cdn.kernel.org (cdn.kernel.org)... 199.232.253.176, 2a04:4e42:fd3::432
Connecting to cdn.kernel.org (cdn.kernel.org)|199.232.253.176|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 133884956 (128M) [application/x-xz]
Saving to: ‘linux-6.0.7.tar.xz’

linux-6.0.7.tar.xz  100%[===================>] 127.68M  1.37MB/s    in 19m 20s

2023-01-09 07:33:21 (113 KB/s) - ‘linux-6.0.7.tar.xz’ saved [133884956/133884956]
```

Step 2*:* Extract the Source Code

When the file is ready, run the tar command to extract the source code:

tar xvf linux-6.0.7.tar.xz

The output displays the extracted kernel source code:

```
ubuntu@ubuntu:~$ tar xvf linux-6.0.7.tar.xzS
linux-6.0.7/virt/
linux-6.0.7/virt/Makefile
linux-6.0.7/virt/kvm/
linux-6.0.7/virt/kvm/Kconfig
linux-6.0.7/virt/kvm/Makefile.kvm
linux-6.0.7/virt/kvm/async_pf.c
linux-6.0.7/virt/kvm/async_pf.h
linux-6.0.7/virt/kvm/binary_stats.c
linux-6.0.7/virt/kvm/coalesced_mmio.c
linux-6.0.7/virt/kvm/coalesced_mmio.h
linux-6.0.7/virt/kvm/dirty_ring.c
linux-6.0.7/virt/kvm/eventfd.c
linux-6.0.7/virt/kvm/irqchip.c
linux-6.0.7/virt/kvm/kvm_main.c
linux-6.0.7/virt/kvm/kvm_mm.h
linux-6.0.7/virt/kvm/pfncache.c
linux-6.0.7/virt/kvm/vfio.c
linux-6.0.7/virt/kvm/vfio.h
linux-6.0.7/virt/lib/
linux-6.0.7/virt/lib/Kconfig
linux-6.0.7/virt/lib/Makefile
linux-6.0.7/virt/lib/irqbypass.c
```

Step 3: Install Required Packages

Install additional packages before building a kernel. To do so, run this command:

sudo apt-get install git fakeroot build-essential ncurses-dev xz-utils libssl-dev bc flex libelf-dev bison

The command we used above installs the essential packages for performing Linux kernel compilation

```
ubuntu@ubuntu:~/linux-6.0.7$ sudo apt-get -f install
[sudo] password for ubuntu:
Reading package lists... Done
Building dependency tree
Reading state information... Done
0 upgraded, 0 newly installed, 0 to remove and 285 not upgraded.
ubuntu@ubuntu:~/linux-6.0.7$ sudo apt-get install build-essential
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following additional packages will be installed:
  binutils binutils-common binutils-x86-64-linux-gnu cpp-9 dpkg-dev fakeroot
  g++ g++-9 gcc gcc-9 gcc-9-base libalgorithm-diff-perl
  libalgorithm-diff-xs-perl libalgorithm-merge-perl libasan5 libatomic1
  libbinutils libc-dev-bin libc6 libc6-dbg libc6-dev libcrypt-dev
  libctf-nobfd0 libctf0 libdpkg-perl libfakeroot libgcc-9-dev libitm1 liblsan0
  libquadmath0 libstdc++-9-dev libtsan0 libubsan1 linux-libc-dev make
  manpages-dev
```

```
ubuntu@ubuntu:~/linux-6.0.7$ sudo apt-get install git fakeroot build-essential ncurses-dev xz-utils libssl-dev bc flex libelf-dev bison
Reading package lists... Done
Building dependency tree
Reading state information... Done
Note, selecting 'libncurses-dev' instead of 'ncurses-dev'
bc is already the newest version (1.07.1-2build1).
bc set to manually installed.
fakeroot is already the newest version (1.24-1).
fakeroot set to manually installed.
build-essential is already the newest version (12.8ubuntu1.1).
The following additional packages will be installed:
  git-man liberror-perl libfl-dev libfl2 libsigsegv2 libssl1.1 m4 zlib1g
  zlib1g-dev
Suggested packages:
  bison-doc flex-doc git-daemon-run | git-daemon-sysvinit git-doc git-el
  git-email git-gui gitk gitweb git-cvs git-mediawiki git-svn ncurses-doc
  libssl-doc m4-doc
The following NEW packages will be installed:
  bison flex git git-man libelf-dev liberror-perl libfl-dev libfl2
  libncurses-dev libsigsegv2 libssl-dev m4 zlib1g-dev
The following packages will be upgraded:
```

Step 4: Configure Kernel

The Linux kernel source code comes with the default configuration. However, you can adjust it to your needs. To do so, follow the steps below:

1. Navigate to the linux-6.0.7 directory using the cd command:

cd linux-6.0.7

2. Copy the existing configuration file using the cp command:

cp -v /boot/config-$(uname -r) .config

```
marko@pnap:~$ cd linux-6.0.7/
marko@pnap:~/linux-6.0.7$ cp -v /boot/config-$(uname -r) .config
'/boot/config-5.15.0-52-generic' -> '.config'
marko@pnap:~/linux-6.0.7$
```

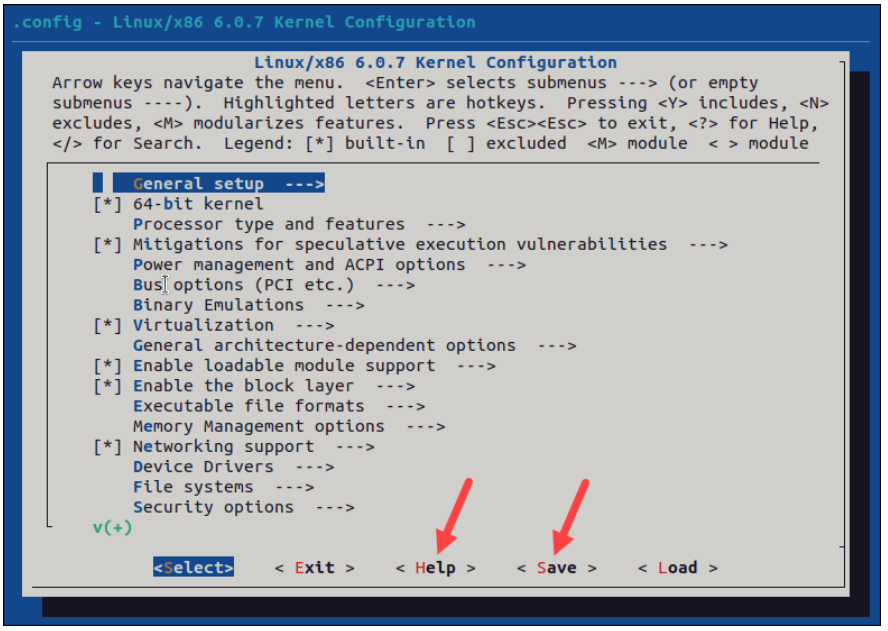
3. To make changes to the configuration file, run the make command:

make menuconfig

The command launches several scripts that open the configuration menu:

```
ubuntu@ubuntu:~/linux-6.0.7$ make menuconfig
  UPD     scripts/kconfig/mconf-cfg
  HOSTCC  scripts/kconfig/mconf.o
  HOSTCC  scripts/kconfig/lxdialog/checklist.o
  HOSTCC  scripts/kconfig/lxdialog/inputbox.o
  HOSTCC  scripts/kconfig/lxdialog/menubox.o
  HOSTCC  scripts/kconfig/lxdialog/textbox.o
  HOSTCC  scripts/kconfig/lxdialog/util.o
  HOSTCC  scripts/kconfig/lxdialog/yesno.o
  HOSTCC  scripts/kconfig/confdata.o
  HOSTCC  scripts/kconfig/expr.o
  LEX     scripts/kconfig/lexer.lex.c
  YACC    scripts/kconfig/parser.tab.[ch]
  HOSTCC  scripts/kconfig/lexer.lex.o
  HOSTCC  scripts/kconfig/menu.o
  HOSTCC  scripts/kconfig/parser.tab.o
  HOSTCC  scripts/kconfig/preprocess.o
  HOSTCC  scripts/kconfig/symbol.o
  HOSTCC  scripts/kconfig/util.o
  HOSTLD  scripts/kconfig/mconf
.config:434:warning: symbol value 'm' invalid for I8K
.config:1998:warning: symbol value 'm' invalid for MCTP
.config:8858:warning: symbol value 'm' invalid for VIDEO_ZORAN_DC30
.config:8859:warning: symbol value 'm' invalid for VIDEO_ZORAN_ZR36060
.config:8860:warning: symbol value 'm' invalid for VIDEO_ZORAN_BUZ
.config:8861:warning: symbol value 'm' invalid for VIDEO_ZORAN_DC10
.config:8862:warning: symbol value 'm' invalid for VIDEO_ZORAN_LML33
.config:8863:warning: symbol value 'm' invalid for VIDEO_ZORAN_LML33R10
.config:8864:warning: symbol value 'm' invalid for VIDEO_ZORAN_AVS6EYES
.config:9955:warning: symbol value 'm' invalid for ANDROID_BINDER_IPC
.config:9956:warning: symbol value 'm' invalid for ANDROID_BINDERFS
configuration written to .config
```

4. The configuration menu includes options such as firmware, file system, network, and memory settings. Use the arrows to make a selection or choose Help to learn more about the options. When you finish making the changes, select Save, and then exit the menu.



Note:

Changing settings for some options can lead to a non-functional kernel. If you are unsure what to change, leave the default settings.

Step 5*:* Build the Kernel

1. Start building the kernel by running the following command:

make

- The process of building and compiling the Linux kernel takes some time to complete.
- The terminal lists all Linux kernel components: memory management, hardware device drivers, filesystem drivers, network drivers, and process management.

```
marko@pnap:~/linux-6.0.7$ make
  SYNC    include/config/auto.conf.cmd
  HOSTCC  scripts/kconfig/conf.o
  HOSTLD  scripts/kconfig/conf
  SYSHDR  arch/x86/include/generated/uapi/asm/unistd_32.h
  SYSHDR  arch/x86/include/generated/uapi/asm/unistd_64.h
  SYSHDR  arch/x86/include/generated/uapi/asm/unistd_x32.h
  SYSTBL  arch/x86/include/generated/asm/syscalls_32.h
  SYSHDR  arch/x86/include/generated/asm/unistd_32_ia32.h
  SYSHDR  arch/x86/include/generated/asm/unistd_64_x32.h
  SYSTBL  arch/x86/include/generated/asm/syscalls_64.h
  HYPERCALLS arch/x86/include/generated/asm/xen-hypercalls.h
  HOSTCC  arch/x86/tools/relocs_32.o
  HOSTCC  arch/x86/tools/relocs_64.o
  HOSTCC  arch/x86/tools/relocs_common.o
  HOSTLD  arch/x86/tools/relocs
  HOSTCC  scripts/genksyms/genksyms.o
  YACC    scripts/genksyms/parse.tab.[ch]
```

If you are compiling the kernel on Ubuntu, you may receive the following error that interrupts the building process:

No rule to make target 'debian/canonical-certs.pem

Disable the conflicting security certificates by executing the two commands below:

scripts/config --disable SYSTEM_TRUSTED_KEYS

scripts/config --disable SYSTEM_REVOCATION_KEYS

The commands return no output. Start the building process again with make, and press Enter repeatedly to confirm the default options for the generation of new certificates.

2. Install the required modules with this command:

sudo make modules_install

```
marko@pnap:~/linux-6.0.7$ sudo make modules_install
  INSTALL sound/usb/line6/snd-usb-line6.ko
  INSTALL sound/usb/line6/snd-usb-pod.ko
  INSTALL sound/usb/line6/snd-usb-podhd.ko
  INSTALL sound/usb/line6/snd-usb-toneport.ko
  INSTALL sound/usb/line6/snd-usb-variax.ko
  INSTALL sound/usb/misc/snd-ua101.ko
  INSTALL sound/usb/snd-usb-audio.ko
  INSTALL sound/usb/snd-usbmidi-lib.ko
  INSTALL sound/usb/usx2y/snd-usb-us122l.ko
  INSTALL sound/usb/usx2y/snd-usb-usx2y.ko
  INSTALL sound/x86/snd-hdmi-lpe-audio.ko
  INSTALL sound/xen/snd_xen_front.ko
  DEPMOD  6.0.7
marko@pnap:~/linux-6.0.7$
```

3. Finally, install the kernel by typing:

sudo make install

The output shows done when finished:

```
marko@pnap:~/linux-6.0.7$ sudo make install
sh ./arch/x86/boot/install.sh 6.0.7 arch/x86/boot/bzImage \
        System.map "/boot"
run-parts: executing /etc/kernel/postinst.d/apt-auto-removal 6.0.7 /boot/vmlinuz-6.0.7
run-parts: executing /etc/kernel/postinst.d/dkms 6.0.7 /boot/vmlinuz-6.0.7
 * dkms: running auto installation service for kernel 6.0.7                    [ OK ]
run-parts: executing /etc/kernel/postinst.d/initramfs-tools 6.0.7 /boot/vmlinuz-6.0.7
update-initramfs: Generating /boot/initrd.img-6.0.7
run-parts: executing /etc/kernel/postinst.d/update-notifier 6.0.7 /boot/vmlinuz-6.0.7
run-parts: executing /etc/kernel/postinst.d/zz-update-grub 6.0.7 /boot/vmlinuz-6.0.7
Sourcing file `/etc/default/grub'
Sourcing file `/etc/default/grub.d/init-select.cfg'
Generating grub configuration file ...
done
marko@pnap:~/linux-6.0.7$
```

Step 6*:* Update the Bootloader (Optional)

The GRUB bootloader is the first program that runs when the system powers on.

The make install command performs this process automatically, but you can also do it manually.

1. Update the initramfs to the installed kernel version:

sudo update-initramfs -c -k 6.0.7

2. Update the GRUB bootloader with this command:

sudo update-grub

The terminal prints out the process and confirmation message:

```
marko@pnap:~/linux-6.0.7$ sudo update-initramfs -c -k 6.0.7
update-initramfs: Generating /boot/initrd.img-6.0.7
marko@pnap:~/linux-6.0.7$ sudo update-grub
Sourcing file `/etc/default/grub'
Sourcing file `/etc/default/grub.d/init-select.cfg'
Generating grub configuration file ...
Found linux image: /boot/vmlinuz-6.0.7
Found initrd image: /boot/initrd.img-6.0.7
Found memtest86+ image: /boot/memtest86+.elf
Found memtest86+ image: /boot/memtest86+.bin
done
```

Step 7*:* Reboot and Verify Kernel Version

When you complete the steps above, reboot the machine.

When the system boots up, verify the kernel version using the uname command:

uname -mrs

The terminal prints out the current Linux kernel version.

```
marko@pnap:~$ uname -mrs
Linux 6.0.7 x86_64
marko@pnap:~$
```

Result:

Thus, the procedure was followed and the kernel file of Linux OS was updated.

EX.NO: 2	UTILIZATION OF GNU TOOLCHAINS FOR EFFECTIVE SYSTEM PROGRAMMING
DATE:	

Aim:

- To develop a C program and execute various GNU Bin Utils tools.
- To debug a C program using GDB tools in a Linux system.

Software requirement:

- Ubuntu Linux distros
- Text editors like gedit, vi in Linux with gcc

Procedure:

- Develop a C program main.c in gedit text editor.
- Create C files mul.c, div.c in gedit text editor which contains the prototype of the main program module main.c
- Create a header with .h in gedit and insert the prototype signatures into the header file head.h.
- Include the header head.h in main.c program.
- After the creation of these files, start building the executable files as follows in the terminal.

```
// Creation of object files
$ gcc –c mul.c
$ gcc –c div.c
$ gcc –i main.c
```

- Create a static library using GNU Bin Utils tool (ar)

```
$ ar rs libhead.a mul.o div.o
```

- Link the object files to create executable file

```
$ gcc -o main main.o libhead.a (or)
$ gcc -o pattern -L . pattern.o -I pattern.a
```

- Apply different GNU tool chains [GDB, Bin Utils (objdump, nm, strings, strips)] and observe the generated output from the terminal window.

Program:

Program for Head.h:

```
int mul(int,int);
int div(int,int);
```

Program for mul.c:

```
int mul(a,b)
{
        return(a*b);
}
```

Program for div.c:

```
int div(c,d)
{
        return(c/d);
}
```

Program for main.c #

```
include <stdio.h> #
include "Head.h" void
main()
{
        int x;
        int y;
        printf("Enter x,y \n");
        scanf("%d %d", &x,&y);
        printf("%d",mul(x,y));
        printf("%d",div(x,y));
}
```

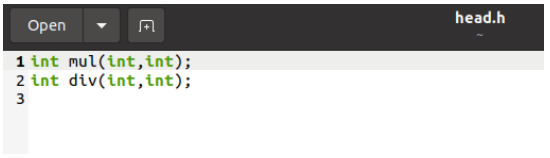
Commands:

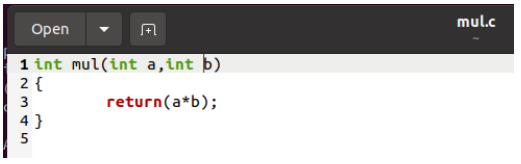
- $ gcc -c mul.c
- $ gcc -c div.c
- $ gcc -c main.c
- $ ar rs libhead.a mul.o div.o
- $ gcc -o main main.o libhead.a
- objdump main.o
- strings main.o
- size main.o
- nm main.o
- strip main.o

Compile the program and execute the following steps:

- gcc -g filename.c
- gdb -q a.out

Output:





GOKULNATH A - 71812301060

div.c

```c
int div(int c,int d)
{
        return(c/d);
}

```

```
ubuntu@ubuntu:~$ strip main.o
ubuntu@ubuntu:~$ size main.o
   text    data     bss     dec     hex filename
    335       0       0     335     14f main.o
```

main.c

```c
#include <stdio.h>
#include "head.h"
void main()
{
        int x; int y;
        printf("Enter x,y:");
        scanf("%d %d", &x,&y);
        printf("Multiplication of x and y is %d\n",mul(x,y));
        printf("Division of x and y is %d\n",div(x,y));
}

```

```
ubuntu@ubuntu:~$ gedit head.h
ubuntu@ubuntu:~$ gedit mul.c
ubuntu@ubuntu:~$ gedit div.c
ubuntu@ubuntu:~$ gedit main.c
ubuntu@ubuntu:~$ gcc -c mul.c
ubuntu@ubuntu:~$ gcc -c div.c
ubuntu@ubuntu:~$ gcc -c main.c
ubuntu@ubuntu:~$ ar rs libhead.a mul.o div.o
ubuntu@ubuntu:~$ gcc -o main main.o libhead.a
```

```
ubuntu@ubuntu:~$ strings main.o
Enter x,y:
%d %d
Multiplication of x and y is %d
Division of x and y is %d
GCC: (Ubuntu 9.4.0-1ubuntu1~20.04.1) 9.4.0
main.c
main
_GLOBAL_OFFSET_TABLE_
printf
__isoc99_scanf
__stack_chk_fail
.symtab
.strtab
.shstrtab
.rela.text
.data
.bss
.rodata
.comment
.note.GNU-stack
.note.gnu.property
.rela.eh_frame
ubuntu@ubuntu:~$ nm main.o
                 U div
                 U _GLOBAL_OFFSET_TABLE_
                 U __isoc99_scanf
0000000000000000 T main
                 U mul
                 U printf
                 U __stack_chk_fail
```

```
ubuntu@ubuntu:~$ objdump -t main

main:     file format elf64-x86-64

SYMBOL TABLE:
0000000000000318 l    d  .interp        0000000000000000              .interp
0000000000000338 l    d  .note.gnu.property     0000000000000000              .note.gnu.property
0000000000000358 l    d  .note.gnu.build-id     0000000000000000              .note.gnu.build-id
000000000000037c l    d  .note.ABI-tag  0000000000000000              .note.ABI-tag
00000000000003a0 l    d  .gnu.hash      0000000000000000              .gnu.hash
00000000000003c8 l    d  .dynsym        0000000000000000              .dynsym
00000000000004b8 l    d  .dynstr        0000000000000000              .dynstr
0000000000000574 l    d  .gnu.version   0000000000000000              .gnu.version
0000000000000588 l    d  .gnu.version_r 0000000000000000              .gnu.version_r
00000000000005c8 l    d  .rela.dyn      0000000000000000              .rela.dyn
0000000000000688 l    d  .rela.plt      0000000000000000              .rela.plt
0000000000001000 l    d  .init  0000000000000000              .init
0000000000001020 l    d  .plt   0000000000000000              .plt
0000000000001060 l    d  .plt.got       0000000000000000              .plt.got
0000000000001070 l    d  .plt.sec       0000000000000000              .plt.sec
00000000000010a0 l    d  .text  0000000000000000              .text
00000000000012d8 l    d  .fini  0000000000000000              .fini
0000000000002000 l    d  .rodata        0000000000000000              .rodata
000000000000205c l    d  .eh_frame_hdr  0000000000000000              .eh_frame_hdr
00000000000020b0 l    d  .eh_frame      0000000000000000              .eh_frame
0000000000003da8 l    d  .init_array    0000000000000000              .init_array
0000000000003db0 l    d  .fini_array    0000000000000000              .fini_array
0000000000003db8 l    d  .dynamic       0000000000000000              .dynamic
0000000000003fa8 l    d  .got   0000000000000000              .got
0000000000004000 l    d  .data  0000000000000000              .data
0000000000004010 l    d  .bss   0000000000000000              .bss
0000000000000000 l    d  .comment       0000000000000000              .comment
0000000000000000 l    df *ABS*  0000000000000000              crtstuff.c
00000000000010d0 l     F .text  0000000000000000              deregister_tm_clones
0000000000001100 l     F .text  0000000000000000              register_tm_clones
0000000000001140 l     F .text  0000000000000000              __do_global_dtors_aux
0000000000004010 l     O .bss   0000000000000001              completed.8061
```

reverse.c

```c
#include<stdio.h>
int main()
{
        int n,rev=0;
        printf("Enter the number:");
        scanf("%d",&n);
        while(n!=0)
        {
                rev=rev*10+(n%10);
                n/=10;
        }
        printf("Enter the reverse number=%d\n",rev);
        return 0;
}
```

```
ubuntu@ubuntu:~$ gedit reverse.c
ubuntu@ubuntu:~$ gcc -g reverse.c
ubuntu@ubuntu:~$ gdb -q a.out
Reading symbols from a.out...
(gdb) r
Starting program: /home/ubuntu/a.out
Enter the number:1234
Enter the reverse number=4321
[Inferior 1 (process 4751) exited normally]
(gdb)
```

Result:

Thus, different GNU tools were applied for the developed ‘C’ program and the output from the terminal window was observed and verified successfully.

EX NO:3	DEBUGGING PROGRAMS USING CSCOPE
DATE:	

AIM:

To debug the programs using cscope.

ALGORITHM:

STEP 1: Install cscope tool in the linux systems.

STEP 2: Create a c program file.

Hell0.c

STEP 3: write the C program with the funuctions like add function.

STEP 4: save the file.

STEP 5: generate cscope database using the command

Cscope -R

STEP 6: open cscope interface using

Cscope -d

STEP 7: Use the following options to debug

1.find function definition

2.find function calls

3.find symbol

4. find text string

STEP 8: trace the add() function.

STEP 9: check where variables are declared and used.

STEP 10: identify logical or syntax errors.

STEP 11: correct and recompile the program using

Gcc add.c -o add

STEP 12: Execute

Cscope hello.c

STEP 13: verify the output.

PROGRAM:

```
#include <stdio.h>
 Int add(int a,intb){
  int res;
  res=a+b;
  return res;
  }
   int main() {
      int n1,n2,res;
      n1=5,n2=10;
      res=add(n1,n2);
      printf("sum=%d\n",res);
      return 0;
}
```

OUTPUT:

RESULT:

Thus debugging the program using cscope has been completed successfully.

EX.NO: 4	CONSTRUCT CHARACTER ORIENTED DEVICE DRIVERS
DATE:	

AIM:

To construct a simple character device driver program in Linux.

CODE:

hello.c

```
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/init.h>

static int __init hello_start(void)
{
    printk(KERN_INFO "Hello, world!\n");
    return 0;
}

static void __exit hello_end(void)
{
    printk(KERN_INFO "Goodbye, world!\n");
}

module_init(hello_start);
module_exit(hello_end);

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Dhinesh-V");
MODULE_DESCRIPTION("Simple Hello World Module");
```

Makefile:

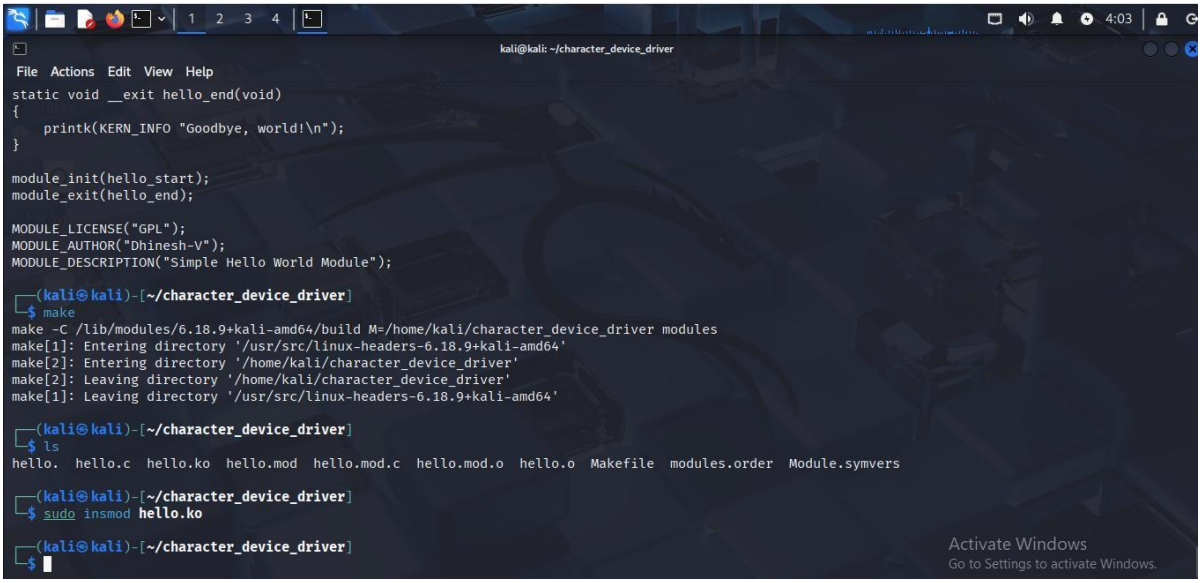
```
obj-m += hello.o
KVERSION = $(shell uname -r)

all:
	make -C /lib/modules/$(KVERSION)/build M=$(PWD) modules

clean:
	make -C /lib/modules/$(KVERSION)/build M=$(PWD) clean
```

COMMANDS:

```
┌──(kali㉿kali)-[~]
└─$ sudo dpkg --configure -a

┌──(kali㉿kali)-[~]
└─$ sudo apt --fix-broken install

┌──(kali㉿kali)-[~]
└─$ sudo apt clean && sudo apt update

┌──(kali㉿kali)-[~]
└─$ sudo apt install linux-image-amd64 linux-headers-amd64

┌──(kali㉿kali)-[~]
└─$ sudo grub-update

┌──(kali㉿kali)-[~]
└─$ sudo reboot

┌──(kali㉿kali)-[~]
└─$ nano hello.c

┌──(kali㉿kali)-[~]
└─$ nano Makefile

┌──(kali㉿kali)-[~]
└─$ make
```

```
┌──(kali㉿kali)-[~]
└─$ sudo insmod hello.ko

┌──(kali㉿kali)-[~]
└─$ lsmod

┌──(kali㉿kali)-[~]
└─$ lsmod | grep hello

┌──(kali㉿kali)-[~]
└─$ dmesg | tail

┌──(kali㉿kali)-[~]
└─$ sudo rmmod hello

┌──(kali㉿kali)-[~]
└─$ dmesg | tail

┌──(kali㉿kali)-[~]
└─$ modinfo hello.ko
```

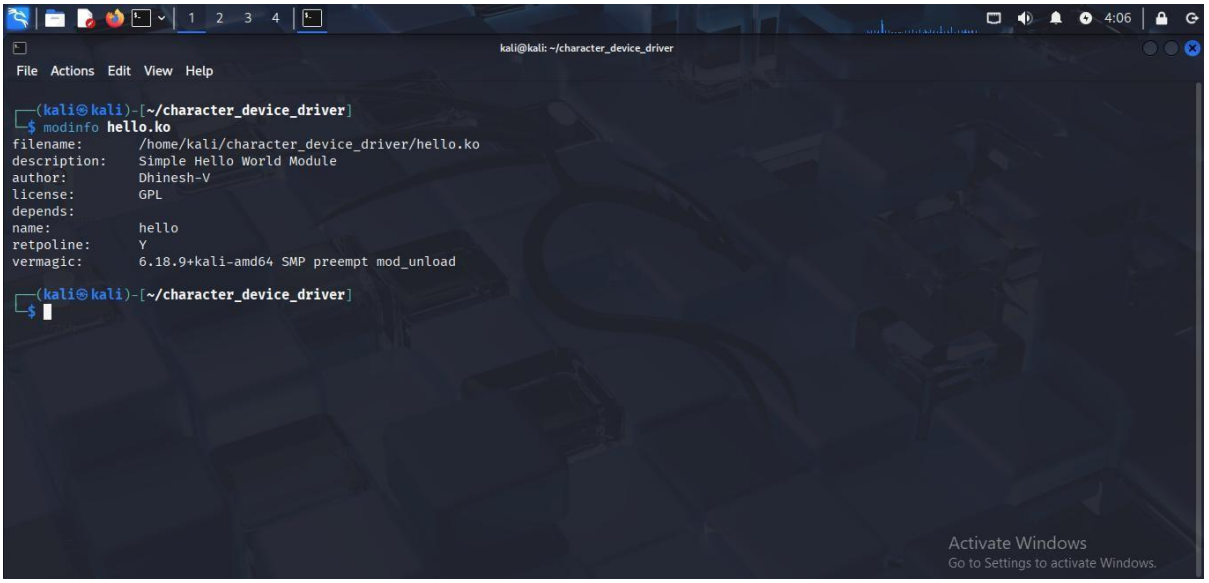
OUTPUT:



```
(kali㉿kali)-[~/character_device_driver]
└─$ lsmod
Module                  Size  Used by
hello                  12288  0
snd_seq_dummy          12288  0
snd_hrtimer            12288  1
snd_seq               114688  7 snd_seq_dummy
snd_seq_device         16384  1 snd_seq
qrtr                   57344  2
rfkill                 45056  2
vboxsf                 49152  0
sunrpc                868352  1
binfmt_misc            28672  1
intel_rapl_msr         20480  0
intel_rapl_common      53248  1 intel_rapl_msr
intel_uncore_frequency_common    16384  0
intel_pmc_core        155648  0
pmt_telemetry          16384  1 intel_pmc_core
pmt_discovery          16384  1 pmt_telemetry
pmt_class              20480  2 pmt_telemetry,pmt_discovery
snd_intel8x0           57344  1
intel_pmc_ssram_telemetry    16384  1 intel_pmc_core
snd_ac97_codec        208896  1 snd_intel8x0
intel_vsec             28672  2 intel_pmc_ssram_telemetry,pmt_telemetry
rapl                   20480  0
ac97_bus               12288  1 snd_ac97_codec
snd_pcm               212992  2 snd_intel8x0,snd_ac97_codec
snd_timer              49152  3 snd_seq,snd_hrtimer,snd_pcm
snd                   159744  10 snd_seq,snd_seq_device,snd_intel8x0,snd_timer,snd_ac97_codec,snd_pcm
joydev                 32768  0
vboxguest              53248  6 vboxsf
pcspkr                 12288  0
soundcore              16384  1 snd
```

```
(kali㉿kali)-[~/character_device_driver]
└─$ lsmod | grep hello
hello                  12288  0

(kali㉿kali)-[~/character_device_driver]
└─$
```

```
(kali㉿kali)-[~/character_device_driver]
└─$ dmesg | tail
[  375.426452] hello: module verification failed: signature and/or required key missing - tainting kernel
[  375.429586] Hello, world!
[  393.447866] 22:33:17.697077 SHCLX11   Shared Clipboard: Converting VBox formats 'BITMAP' to 'INVALID' for X11 (idxFmtX11=0, fmtX11=0, atomT
[  393.450293] 22:33:17.699546 SHCLX11   Shared Clipboard: Converting VBox formats 'BITMAP' to 'INVALID' for X11 (idxFmtX11=0, fmtX11=0, atomT
[  393.451503] 22:33:17.702088 SHCLX11   Shared Clipboard: Converting VBox formats 'BITMAP' to 'INVALID' for X11 (idxFmtX11=0, fmtX11=0, atomT
[  449.596993] 22:34:13.874225 SHCLX11   Shared Clipboard: Converting VBox formats 'BITMAP' to 'INVALID' for X11 (idxFmtX11=0, fmtX11=0, atomT
[  449.599632] 22:34:13.876838 SHCLX11   Shared Clipboard: Converting VBox formats 'BITMAP' to 'INVALID' for X11 (idxFmtX11=0, fmtX11=0, atomT
[  449.600881] 22:34:13.879706 SHCLX11   Shared Clipboard: Converting VBox formats 'BITMAP' to 'INVALID' for X11 (idxFmtX11=0, fmtX11=0, atomT
[  486.670428] 22:34:50.966177 SHCLX11   Shared Clipboard: Converting VBox formats 'BITMAP' to 'INVALID' for X11 (idxFmtX11=0, fmtX11=0, atomT
[  486.673363] 22:34:50.969160 SHCLX11   Shared Clipboard: Converting VBox formats 'BITMAP' to 'INVALID' for X11 (idxFmtX11=0, fmtX11=0, atomT

(kali㉿kali)-[~/character_device_driver]
└─$ sudo rmmod hello

(kali㉿kali)-[~/character_device_driver]
└─$ dmesg | tail
[  449.596993] 22:34:13.874225 SHCLX11   Shared Clipboard: Converting VBox formats 'BITMAP' to 'INVALID' for X11 (idxFmtX11=0, fmtX11=0, atomT
[  449.599632] 22:34:13.876838 SHCLX11   Shared Clipboard: Converting VBox formats 'BITMAP' to 'INVALID' for X11 (idxFmtX11=0, fmtX11=0, atomT
[  449.600881] 22:34:13.879706 SHCLX11   Shared Clipboard: Converting VBox formats 'BITMAP' to 'INVALID' for X11 (idxFmtX11=0, fmtX11=0, atomT
[  486.670428] 22:34:50.966177 SHCLX11   Shared Clipboard: Converting VBox formats 'BITMAP' to 'INVALID' for X11 (idxFmtX11=0, fmtX11=0, atomT
[  486.673363] 22:34:50.969160 SHCLX11   Shared Clipboard: Converting VBox formats 'BITMAP' to 'INVALID' for X11 (idxFmtX11=0, fmtX11=0, atomT
[  486.674253] 22:34:50.971683 SHCLX11   Shared Clipboard: Converting VBox formats 'BITMAP' to 'INVALID' for X11 (idxFmtX11=0, fmtX11=0, atomT
[  520.661968] 22:35:24.974696 SHCLX11   Shared Clipboard: Converting VBox formats 'BITMAP' to 'INVALID' for X11 (idxFmtX11=0, fmtX11=0, atomT
[  520.664663] 22:35:24.977400 SHCLX11   Shared Clipboard: Converting VBox formats 'BITMAP' to 'INVALID' for X11 (idxFmtX11=0, fmtX11=0, atomT
[  520.665726] 22:35:24.980131 SHCLX11   Shared Clipboard: Converting VBox formats 'BITMAP' to 'INVALID' for X11 (idxFmtX11=0, fmtX11=0, atomT
[  542.024225] Goodbye, world!

(kali㉿kali)-[~/character_device_driver]
└─$
```



Result:

The C program is written to create Character Device Driver program and output is verified successfully.

EX.NO: 5	IMPLEMENTATION OF TASK MANAGEMENT IN REAL-TIME OPERATING SYSTEMS (RTOS) USING MICROC/OS-II
DATE:	

Aim:

To develop a C program for creating tasks using FreeRTOS APIs.

Software Requirement:

- Ubuntu Linux distros
- Text editors like gedit, vi in linux with gcc.

Procedure:

- Install the dependencies for Ubuntu

  sudo apt-get install libcs6-dev-i386
- Navigate to the C source code

  $cd Project/main.c
- Compile the Makefile using make command

  $make

Program:

```
// Task Creation
#include<stdio.h>
#include<stdlib.h>
#include<stdbool.h>
#include<FreeRTOS.h>
#include<task.h>
void vTask1(void*);
void vTask2(void*);
void vApplicationIdleHook(void);
int main(void)
{
        xTaskCreate(vTask1,"Task1",1000,NULL,1,NULL);
        xTaskCreate(vTask2,"Task2",1000,NULL,1,NULL);
        vTaskStartScheduler();
        return 0;
}
```

```
void vAssertCalled( unsigned long ulLine, const char * const pcFileName )
    {
            taskENTER_CRITICAL();
            {
                    printf("[ASSERT] %s:%lu\n", pcFileName, ulLine);
                    flush(stdout);
            }
            taskEXIT_CRITICAL();
            exit(-1);
    }
    void vTask1(void* parameter)
    {
            while(1)
            {
                    printf("Task
                    1\n");
                    sleep(500);
            }
    }
    void vTask2(void* parameter)
    {
            while(1)
            {
                    printf("Task
                    2\n");
                    sleep(500);
            }
    }
    void vApplicationIdleHook(void)
    {
            //printf("Idle\r\n");
        }
```

Output:

```
Task1
Task2
Task1
Task2
Task1
Task2
Task1
Task2
Task1
Task2
Task1
Task2
Task1
Task2
Task1
Task2
Task1
Task2
Task1
Task2
Task1
Task2
Task1
Task2
Task1
Task2
Task1
Task2
Task1
Task2
Task1
Task2
```

Result:

Thus, task creation using FreeRTOS API functions is executed and the output is verified successfully.

EX NO:6	IMPLEMENTATION OF INTERUPPT MANAGEMENT IN REAL-TIME OPERATING SYSTEMS (RTOS) USING MICROC/OS-II
DATE:	

AIM:

To implement interrupt management in real time operating system using micro/os II.

ALGORITHM:

Step 1: Initialize the tasks (vTask1, vTask2, vTask3, vTask4) using xTaskCreate()
Step 2: Start the scheduler using vTaskStartScheduler()
Step 3: Execute Task 1 and Task 2 repeatedly with 250 ms delay using vTaskDelay()
Step 4: Execute Task 3 (250 ms) and Task 4 (500 ms) periodically using vTaskDelayUntil()
Step 5: Run idle hook when CPU is free and handle errors using vAssertCalled()

PROGRAM:

```
#include <stdio.h>

#include <stdlib.h>

#include <FreeRTOS.h>

#include <task.h>

#include <timers.h>

#define TASKSCHEDULER

#ifdef TASKSCHEDULER

void vTask1(void*);

void vTask2(void*);

void vTask3(void*);

void vTask4(void*);

#endif

void vApplicationIdleHook(void);

int main(void)

{

#ifdef TASKSCHEDULER

xTaskCreate( vTask1, &quot;Task 1&quot;, 1000, NULL, 1, NULL );

xTaskCreate( vTask2, &quot;Task 2&quot;, 1000, NULL, 1, NULL );

xTaskCreate( vTask3, &quot;Task 3&quot;, 1000, NULL, 1, NULL );

xTaskCreate( vTask4, &quot;Task 4&quot;, 1000, NULL, 1, NULL );
```

```
#endif
vTaskStartScheduler();
return 0;
}
void vAssertCalled( unsigned long ulLine, const char * const pcFileName )
{
taskENTER_CRITICAL();


{
printf(&quot;[ASSERT] %s:%lu\n&quot;, pcFileName, ulLine);
flush(stdout);
}
taskEXIT_CRITICAL();
exit(-1);
}
#ifdef TASKSCHEDULER
void vTask1(void* parameter)
{
while(1)
{
printf(&quot;Task 1\n&quot;);
vTaskDelay(pdMS_TO_TICKS(250));
}
}
void vTask2(void* parameter)
{
while(1)
{
printf(&quot;Task 2\n&quot;);
vTaskDelay(pdMS_TO_TICKS(250));
}
```

```
}
void vTask3(void* parameter)
{
TickType_t xLastWaketime = xTaskGetTickCount(); while(1)
{

printf(&quot;Task 3 with 250ms\n&quot;);
vTaskDelayUntil(&amp;xLastWaketime, pdMS_TO_TICKS(250));
}
}
void vTask4(void* parameter)
{
TickType_t xLastWaketime = xTaskGetTickCount();
while(1)
{
printf(&quot;Task 4 with 500ms\n&quot;);
vTaskDelayUntil(&amp;xLastWaketime, pdMS_TO_TICKS(500));
}
}
#endif
void vApplicationIdleHook(void)
{
// printf(&quot;Idle\r\n&quot;);
}
```

OUTPUT:

```
Output

Task 1
Task 2
Task 3 with 250ms
Task 4 with 500ms
```

RESULT:

Thus implementing interrupt management in real time operating system using micro/os II has been completed successfully.

EX NO: 07	**DEVELOPMENT OF BLUETOOTH INTERFACING USING MSP430 LAUNCHPAD**
DATE:	

Aim:

To write a sketch program to connect the Bluetooth Module with MSP430G2553 to control a LED.

Apparatus Required:

- MSP430G2553 Launchpad
- Energia IDE
- HC-05 Bluetooth module.

Procedure:

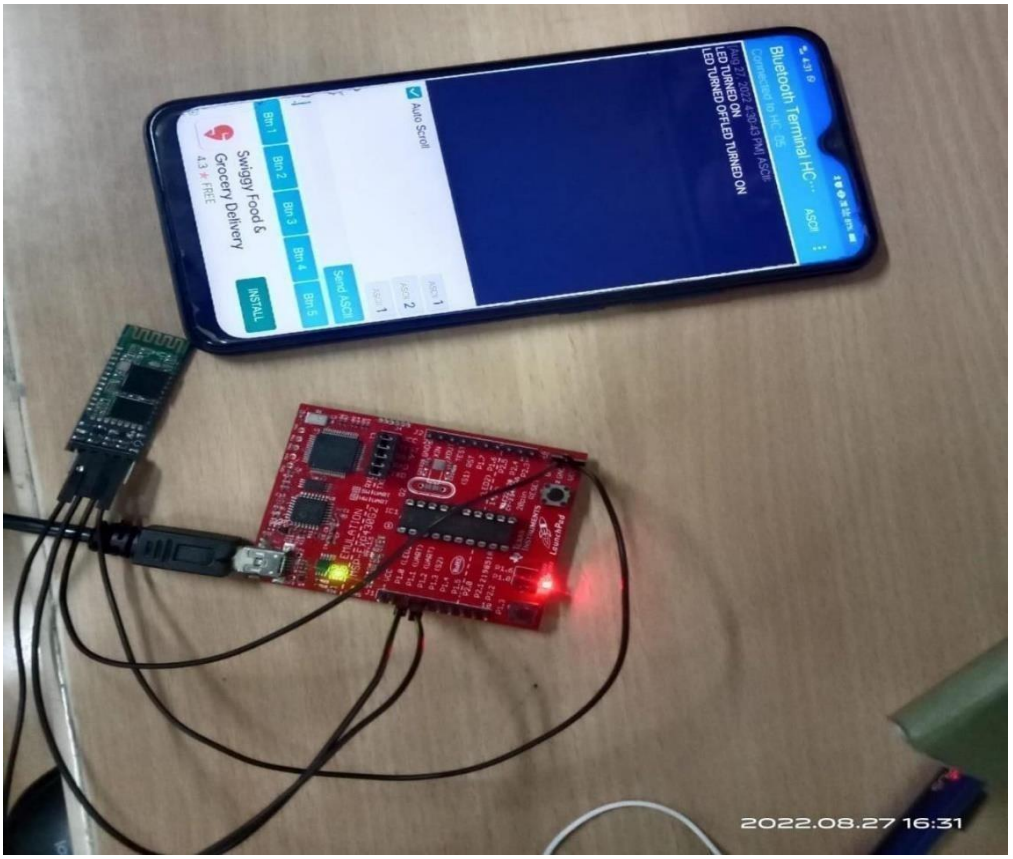
- Attach the MSP430G2553 board with the system.
- Attach the Bluetooth Module with MSP430G2553 board.
- Double click on Energia IDE on the desktop.
- Select the board type as MSP430G2553 Launchpad from Tool.
- Create a new program on Energia IDE and save it.
- Compile the program and upload it to the MSP430G2553 Launchpad board.
- Run the program and verify the output by controlling the LED using an Android application on mobile.

Program:

```
#define LED RED_LED
void setup()
{
        Serial.begin(9600);
        pinMode(2,
        OUTPUT);
}
void loop()
{
        if (Serial.available())
        {
                char data_received;
                data_received = Serial.read();
                if (data_received == '1')
                {
                        digitalWrite(LED, HIGH);
                        Serial.write("LED turned ON\n");
                }
```

```
            if (data_received == '2')
            {
                digitalWrite(LED, LOW);
                Serial.write("LED turned OFF\n");
            }
        }
}
```

Output:



Result:

Thus, the sketch program to connect the Bluetooth Module with MSP430G2553 to control a led was implemented successfully.

EX NO: 08	**DEVELOPMENT OF ESP8266 INTERFACING (WIFI) USING MSP430 LAUNCHPAD**
DATE:	

Aim:

To write a sketch program to connect the ESP8266 WiFi Module with MSP430G2553 to send a data to browser.

Apparatus Required:

- MSP430G2553 Launchpad
- Energia IDE
- ESP8266 WiFi module.

Procedure:

- Attach the MSP430G2553 board with the system.
- Attach the WiFi Module with MSP430G2553 board.
- Double click on Energia IDE on the desktop.
- Select the board type as MSP430G2553 Launchpad from Tool.
- Create a new program on Energia IDE and save it.
- Compile the program and upload it to the MSP430G2553 Launchpad board.
- Run the program and verify the output by sending data to browser.

Program:

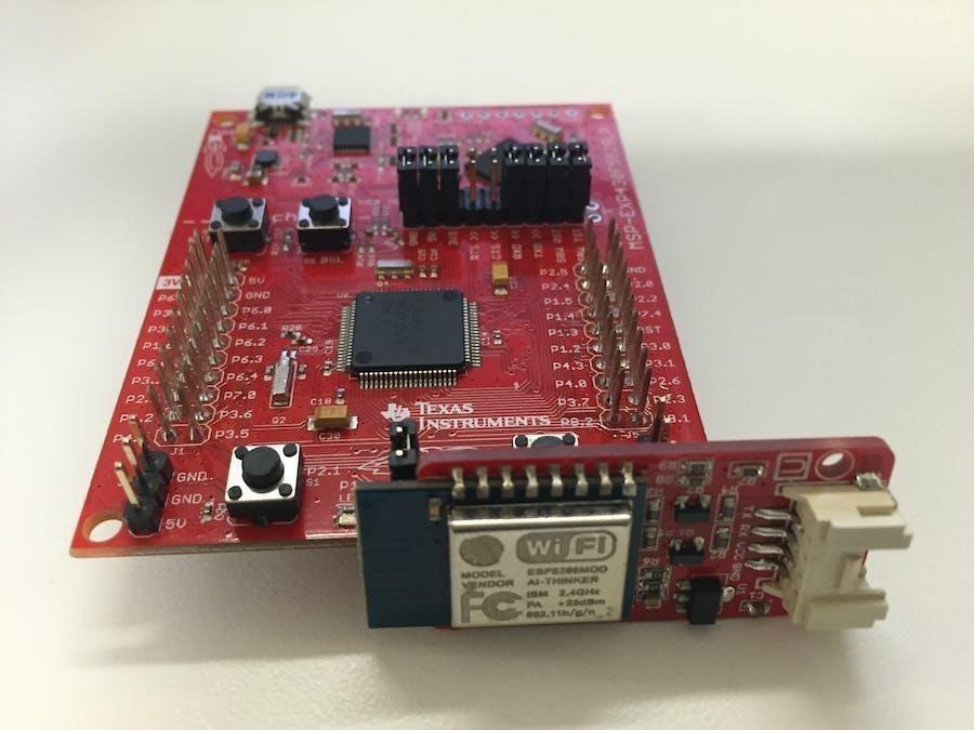
```
#define SSID "RAGHAV"
#define PASS "12345678"
#define DST_IP "things.ubidots.com"
#define idvariable "569fc4ba76254229c49896a6"

int len;

void setup()
{
  char cmd[254];

  // Open serial communication
  Serial.begin(9600);
  Serial.setTimeout(5000);

  // Test if module is ready
  Serial.println("AT+RST");
  delay(1000);

  if (Serial.find("ready"))
  {
    Serial.println("Module is ready");
  }
  else
  {
    Serial.println("Module have no response.");
    while (1);
  }
```

```
  delay(1000);

  // Connect to WiFi
  boolean connected = false;

  for (int i = 0; i < 5; i++)
  {
    if (connectWiFi())
    {
      connected = true;
      break;
    }
  }

  if (!connected)
  {
    while (1);
  }

  delay(5000);

  Serial.println("AT+CIPMUX=0");
}

void loop()
{
  int value = analogRead(A0);
  int num = 0;

  String var = "{\"value\":" + String(value) + "}";
  num = var.length();

  String cmd = "AT+CIPSTART=\"TCP\",\"";
  cmd += DST_IP;
  cmd += "\",80";

  Serial.println(cmd);

  if (Serial.find("Error"))
    return;

  len = strlen("POST /api/v1.6/datasources/");
  len += strlen(idvariable);
  len += strlen("/values HTTP/1.1\nContent-Type: application/json\nContent-Length: ");

  char numlength[4];
  sprintf(numlength, "%d", num);

  len += strlen(numlength);
  len += num;
```

```
  len += strlen(token);
  len += strlen("\nHost: things.ubidots.com\n\n");
  len += strlen("\n\n");

  Serial.print("AT+CIPSEND=");
  Serial.println(len);

  if (Serial.find(">"))
  {
    Serial.print("POST /api/v1.6/variables/");
    Serial.print(idvariable);
    Serial.print("/values HTTP/1.1\nContent-Type: application/json\nContent-Length: ");
    Serial.print(num);
    Serial.print("\nX-Auth-Token: ");
    Serial.print(token);
    Serial.print("\nHost: things.ubidots.com\n\n");
    Serial.print(var);
    Serial.println("\n\n");
  }
  else
  {
    Serial.println("AT+CIPCLOSE");
    delay(1000);
    return;
  }

  delay(9000);

  // Clear input buffer
  while (Serial.available())
  {
    char c = Serial.read();
  }

  delay(1000);
}

boolean connectWiFi()
{
  Serial.println("AT+CWMODE=1");

  String cmd = "AT+CWJAP=\"";
  cmd += SSID;
  cmd += "\",\"";
  cmd += PASS;
  cmd += "\"";

  Serial.println(cmd);
  delay(2000);
```

```
if (Serial.find("OK"))
  {
    return true;
  }
  else
  {
    return false;
  }
}
```

Output:



Result:

Thus, the sketch program to connect a ESP8266 Wifi Module with MSP430G2553 to send a data to browser was implemented successfully.

EX NO:6	IMPLEMENTATION OF INTERUPPT MANAGEMENT IN REAL-TIME OPERATING SYSTEMS (RTOS) USING MICROC/OS-II
DATE:	

AIM:

To implement interrupt management in real time operating system using micro/os II.

ALGORITHM:

Step 1: Initialize the tasks (vTask1, vTask2, vTask3, vTask4) using xTaskCreate()
Step 2: Start the scheduler using vTaskStartScheduler()
Step 3: Execute Task 1 and Task 2 repeatedly with 250 ms delay using vTaskDelay()
Step 4: Execute Task 3 (250 ms) and Task 4 (500 ms) periodically using vTaskDelayUntil()
Step 5: Run idle hook when CPU is free and handle errors using vAssertCalled()

PROGRAM:

```
#include <stdio.h>

#include <stdlib.h>

#include <FreeRTOS.h>

#include <task.h>

#include <timers.h>

#define TASKSCHEDULER

#ifdef TASKSCHEDULER

void vTask1(void*);

void vTask2(void*);

void vTask3(void*);

void vTask4(void*);

#endif

void vApplicationIdleHook(void);

int main(void)

{

#ifdef TASKSCHEDULER

xTaskCreate( vTask1, &quot;Task 1&quot;, 1000, NULL, 1, NULL );

xTaskCreate( vTask2, &quot;Task 2&quot;, 1000, NULL, 1, NULL );

xTaskCreate( vTask3, &quot;Task 3&quot;, 1000, NULL, 1, NULL );

xTaskCreate( vTask4, &quot;Task 4&quot;, 1000, NULL, 1, NULL );
```

```
#endif
vTaskStartScheduler();
return 0;
}
void vAssertCalled( unsigned long ulLine, const char * const pcFileName )
{
taskENTER_CRITICAL();


{
printf(&quot;[ASSERT] %s:%lu\n&quot;, pcFileName, ulLine);
flush(stdout);
}
taskEXIT_CRITICAL();
exit(-1);
}
#ifdef TASKSCHEDULER
void vTask1(void* parameter)
{
while(1)
{
printf(&quot;Task 1\n&quot;);
vTaskDelay(pdMS_TO_TICKS(250));
}
}
void vTask2(void* parameter)
{
while(1)
{
printf(&quot;Task 2\n&quot;);
vTaskDelay(pdMS_TO_TICKS(250));
}
```

```
}
void vTask3(void* parameter)
{
TickType_t xLastWaketime = xTaskGetTickCount(); while(1)
{

printf(&quot;Task 3 with 250ms\n&quot;);
vTaskDelayUntil(&amp;xLastWaketime, pdMS_TO_TICKS(250));
}
}
void vTask4(void* parameter)
{
TickType_t xLastWaketime = xTaskGetTickCount();
while(1)
{
printf(&quot;Task 4 with 500ms\n&quot;);
vTaskDelayUntil(&amp;xLastWaketime, pdMS_TO_TICKS(500));
}
}
#endif
void vApplicationIdleHook(void)
{
// printf(&quot;Idle\r\n&quot;);
}
```

OUTPUT:

```
Output

Task 1
Task 2
Task 3 with 250ms
Task 4 with 500ms
```

RESULT:

Thus implementing interrupt management in real time operating system using micro/os II has been completed successfully.

EX NO: 10	**INTERFACING PUSH BUTTON USING TI CC3200 LAUNCHPAD**
DATE:	

Aim:

To write a CC3200 sketch to turn on and off a light emitting diode (LED) connected to a digital Pin when pressing a push button attached to a digital pin.

Apparatus Required:

- Energia
- CC3200 Board
- Push Button
- 10K ohm resistor
- Breadboard

Procedure:

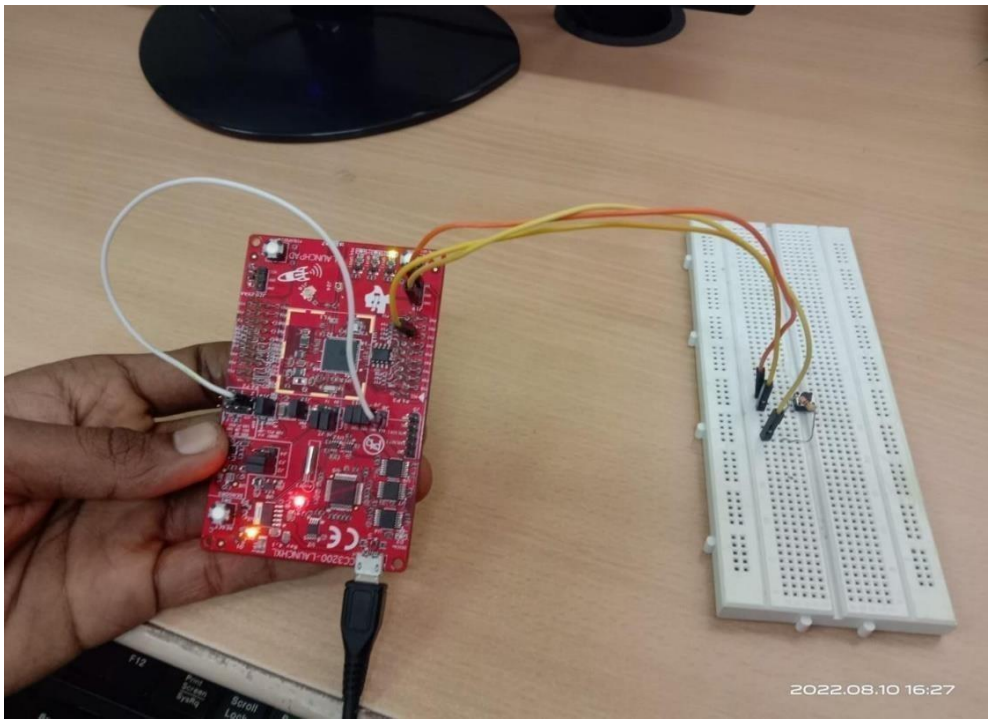
- Attach the CC3200 board to the system.
- Connect Push Button to digital pin 8 and LED with the digital pin 2 of the CC3200 board.
- Double-click on Energia on the desktop.
- Select the board type as CC3200 from Tools-Board and also select the COM port number from the PORT option.
- Create a new program in the Energia software and save it.
- Compile the program and upload it to the CC3200 board.
- Run the program and verify the output.

Code:

```
const int buttonPin = 8;
const int ledPin = 2;

int buttonState = 0;

void setup()
{
  pinMode(ledPin, OUTPUT);
  pinMode(buttonPin, INPUT);
  Serial.begin(9600);
}

void loop()
{
  buttonState = digitalRead(buttonPin);

  if (buttonState == HIGH)
  {
    digitalWrite(ledPin, HIGH);
```

```
Serial.println("LED glows");
  }
  else
  {
    digitalWrite(ledPin, LOW);
  }
}
```

Output:



Result:

Thus, the Energia sketch to interface pushbutton and LED with CC3200 was executed and the output was verified successfully.